# Test Case 1

# University Scholarship Allocation System using Python

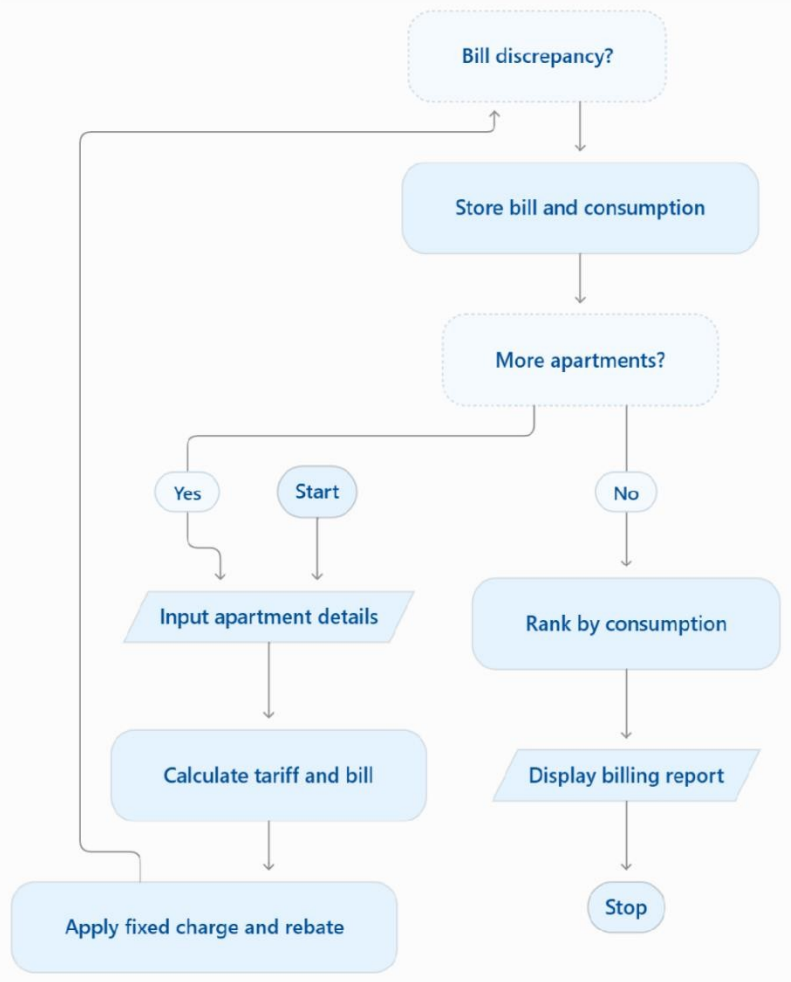
**1. Problem Statement**

The University Scholarship Allocation System evaluates students based on aggregate marks, subject-wise minimum marks, attendance percentage, and family annual income. It identifies eligible students and ranks them according to aggregate marks. If two students have equal marks, attendance is used as the first tie-breaker, followed by lower family income. The system displays the final ranked list of eligible students.

**2. Algorithm**

1. Start the program.
2. Read the number of applicants.
3. Enter each student's name, aggregate marks, subject marks, attendance percentage, and annual family income.
4. Check the eligibility criteria:
    - Aggregate marks must be at least 75%.
    - Every subject mark must be at least 60%.
    - Attendance must be at least 75%.
    - Annual family income must not exceed ₹2,50,000.
5. Add eligible students to the eligible list.
6. Sort eligible students by:
    - Higher aggregate marks first.
    - Higher attendance first if aggregate marks are equal.
    - Lower family income first if both are equal.
7. Display the ranked list of eligible students.
8. If no students qualify, display a suitable message.
9. Stop the program.

**3. Flowchart**




**4. source code**

```
applicants = []
n = int(input("Enter number of applicants: "))


for i in range(n):
    print("\nApplicant", i + 1)
    name = input("Enter student name: ")
    marks = float(input("Enter aggregate marks (%): "))
```

```
    subjects = list(map(float, input(
        "Enter marks in 3 subjects: ").split()))
    attendance = float(input("Enter attendance (%): "))
    income = float(input("Enter annual family income: Rs. "))

    if (marks >= 75 and min(subjects) >= 60
            and attendance >= 75 and income <= 250000):
        applicants.append(
            (name, marks, attendance, income))
        print("Eligible for scholarship")
    else:
        print("Not eligible for scholarship")
applicants.sort(key=lambda x: (-x[1], -x[2], x[3]))

print("\n--- Scholarship Merit List ---")

if applicants:
    for rank, student in enumerate(applicants, 1):
        print(f"{rank}. {student[0]} - Marks: {student[1]}%, "
              f"Attendance: {student[2]}%, Income: Rs.{student[3]}")
else:
    print("No eligible applicants")
```

**5. Sample Input**

```
Enter number of applicants: 3

Enter student name: Ravi
Enter aggregate marks: 85
Enter 3 subject marks: 80 90 75
Enter attendance: 90
Enter family income: 200000

Enter student name: Sita
Enter aggregate marks: 85
Enter 3 subject marks: 75 80 70
Enter attendance: 95
Enter family income: 220000

Enter student name: Aman
Enter aggregate marks: 70
Enter 3 subject marks: 65 70 75
Enter attendance: 80
Enter family income: 150000
```

**6. Sample Output**

```
Eligible for scholarship
Eligible for scholarship
Not eligible for scholarship

--- Scholarship Merit List ---

1 Sita Marks: 85.0 Attendance: 95.0 Income: Rs. 220000.0
2 Ravi Marks: 85.0 Attendance: 90.0 Income: Rs. 200000.0
```

# Test Case 2

# Employee Payroll and Deduction Analysis Using Python

**1. Problem Statement**

The Employee Payroll and Deduction Analysis system calculates an employee's gross salary using basic pay, allowances, overtime, and incentives. It applies progressive deductions based

on salary and generates a formatted salary statement showing gross salary, total deductions, and net salary.

**2. Algorithm**

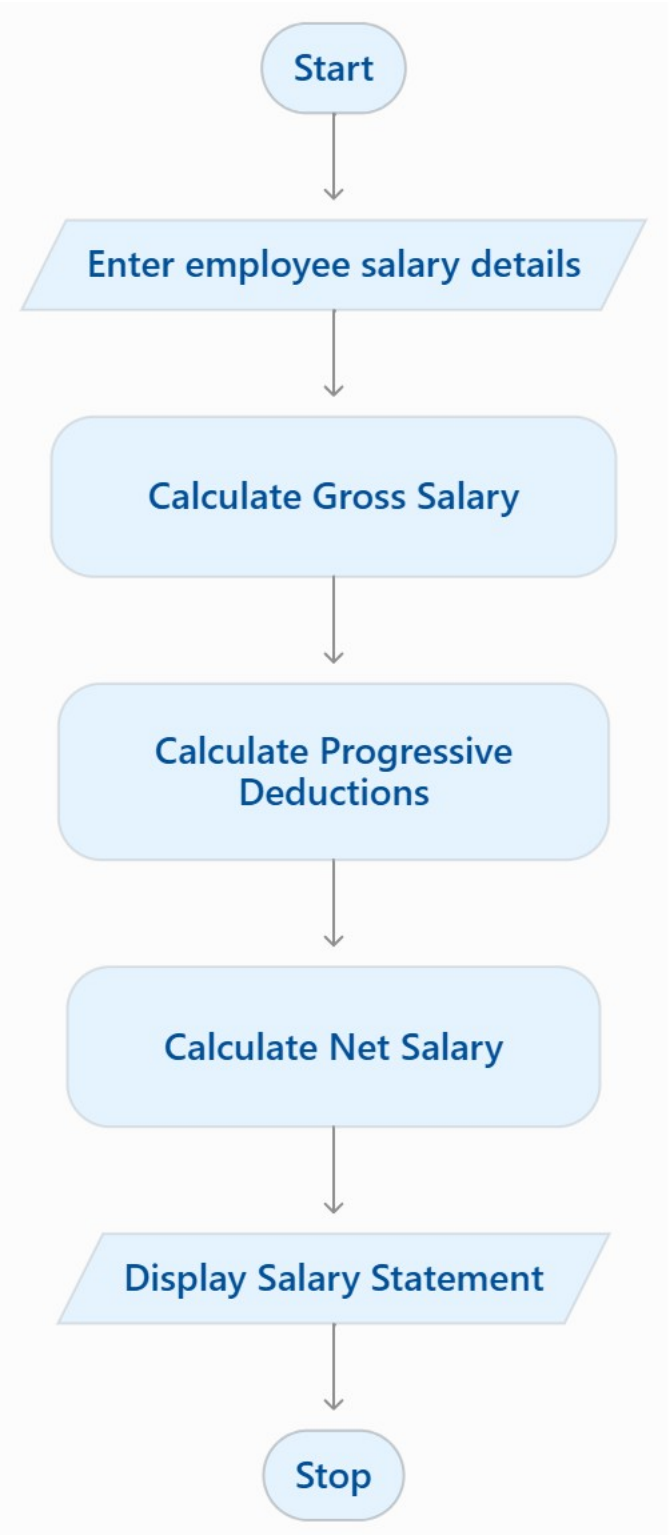
1. Start the program.
2. Enter employee name, basic salary, allowances, overtime pay, and incentives.
3. Calculate gross salary by adding all earnings.
4. Calculate progressive deductions based on gross salary.
5. Calculate net salary by subtracting total deductions from gross salary.
6. Display the formatted salary statement.
7. Stop the program.

Assumed deduction rules for this example:

- First ₹20,000: 5% deduction.
- Next ₹20,000: 10% deduction.
- Amount above ₹40,000: 15% deduction.

These are example rates for the project, not official payroll or tax rules.

**3. Flowchart**




**4.source code**

```
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: Rs. "))
allowances = float(input("Enter allowances: Rs. "))
overtime = float(input("Enter overtime pay: Rs. "))
incentives = float(input("Enter incentives: Rs. "))

gross = basic + allowances + overtime + incentives
if gross <= 20000:
    deduction = gross * 0.05
elif gross <= 40000:
    deduction = 20000 * 0.05 + (gross - 20000) * 0.10
else:
    deduction = (20000 * 0.05
                + 20000 * 0.10
                + (gross - 40000) * 0.15)

net = gross - deduction

print("\n------ SALARY STATEMENT ------")
print("Employee Name :", name)
print("Basic Salary  : Rs.", basic)
print("Allowances    : Rs.", allowances)
print("Overtime Pay  : Rs.", overtime)
print("Incentives    : Rs.", incentives)
print("Gross Salary  : Rs.", gross)
print("Deductions    : Rs.", deduction)
print("Net Salary    : Rs.", net)
```

**5. Sample Input**

```
Enter employee name: Ravi
Enter basic salary: 35000
Enter allowances: 5000
Enter overtime pay: 2000
Enter incentives: 3000
```

**6. Sample Output**

```
------ SALARY STATEMENT ------
Employee Name : Ravi
Basic Salary  : Rs. 35000.0
Allowances    : Rs. 5000.0
Overtime Pay  : Rs. 2000.0
Incentives    : Rs. 3000.0
Gross Salary  : Rs. 45000.0
Deductions    : Rs. 3750.0
Net Salary    : Rs. 41250.0
```

# Test Case 3

# Smart Parking Fee Management System in Python

**1. Problem Statement**

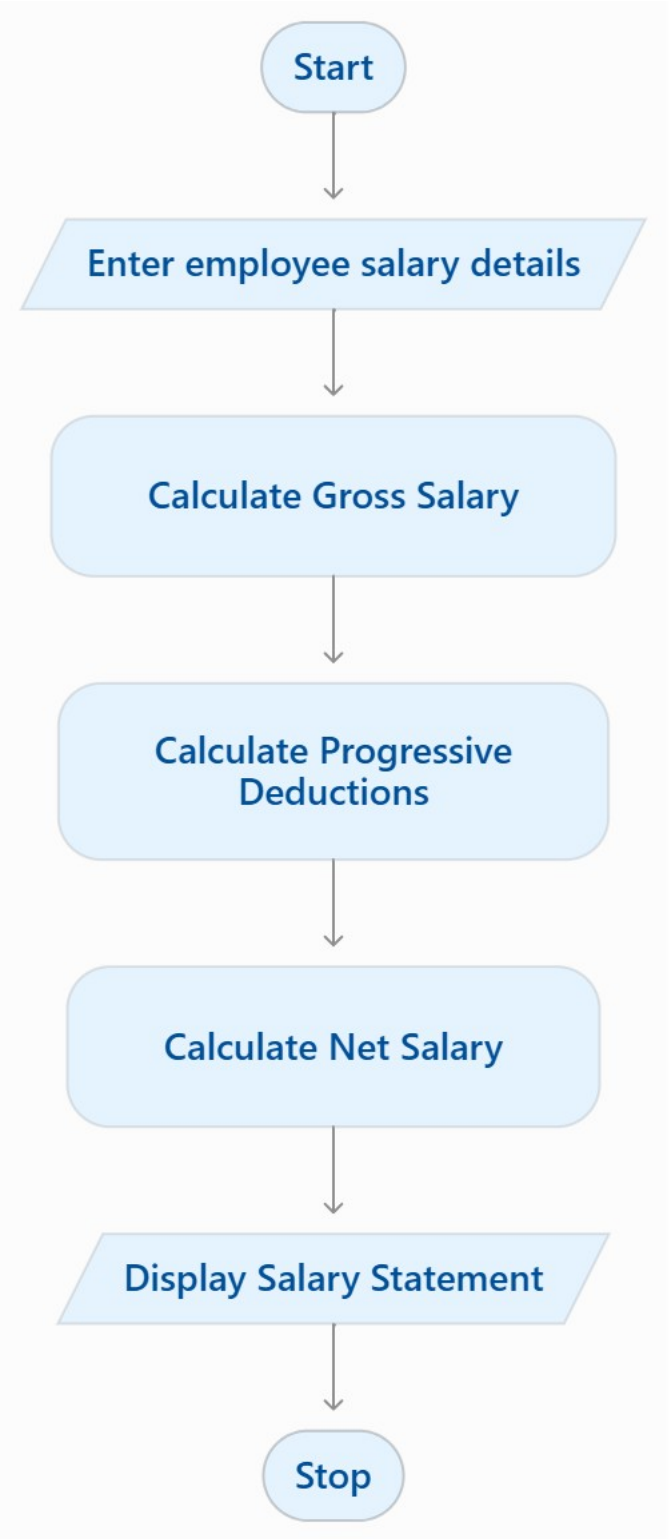
Smart Parking Fee Management: Calculate charges from entry and exit times, vehicle categories, overnight parking, daily limits, and penalty conditions while validating time inputs.

**2. Algorithm**

1. Start the program.
2. Define the hourly parking rates for bikes, cars, and trucks.
3. Read the vehicle category, entry time, and exit time.
4. Validate the vehicle category and time format.
5. Convert the entry and exit times into minutes.
6. If the exit time is earlier than the entry time, treat it as an overnight parking session.
7. Calculate the total parking duration in minutes.

8. Convert the duration into hours, rounding up any partial hour.
9. Calculate the parking fee using the vehicle's hourly rate.
10. If parking extends overnight, add the overnight penalty.
11. If parking exceeds 24 hours, apply the daily maximum limit for each 24-hour period.
12. Display the parking duration, base fee, penalties, and total fee.
13. Stop the program.

**3. Flowchart**




**4.source code**

```
import math
from datetime import datetime
```

```
rates = {"bike": 20, "car": 50, "truck": 80}

try:
    vehicle = input("Enter vehicle (bike/car/truck): ").lower()
    entry = input("Enter entry time (HH:MM): ")
    exit_time = input("Enter exit time (HH:MM): ")

    start = datetime.strptime(entry, "%H:%M")
    end = datetime.strptime(exit_time, "%H:%M")

    if vehicle not in rates:
        raise ValueError("Invalid vehicle category")

    duration = (end.hour * 60 + end.minute) - \
               (start.hour * 60 + start.minute)

    if duration < 0:
        duration += 1440
    elif duration == 0:
        raise ValueError("Entry and exit times cannot be equal")

    hours = math.ceil(duration / 60)
    fee = min(hours * rates[vehicle], 300)

    overnight = end < start
    penalty = 50 if overnight else 0
    total = fee + penalty

    print("\nParking duration:", hours, "hours")
```

```
    print("Base fee: Rs.", fee)
    print("Overnight penalty: Rs.", penalty)
    print("Total parking fee: Rs.", total)

except ValueError as e:
    print("Error:", e)
```

**5. Sample Input**

```
Enter vehicle (bike/car/truck): car
Enter entry time (HH:MM): 21:00
Enter exit time (HH:MM): 02:30
```

**6. Sample Output**

```
Parking duration: 6 hours
Base fee: Rs. 300
Overnight penalty: Rs. 50
Total parking fee: Rs. 350
```

# Test Case 4

# Food Supply Chain Stock Reconciliation Using Python

**1. Problem Statement**

Develop a Python program to manage and reconcile food stock across several locations. The program should accept quantities received, distributed, damaged, and remaining at each location. It should calculate the expected remaining stock, identify discrepancies between expected and actual stock, and determine the replenishment quantity required to meet the minimum stock level.

**2. Algorithm**

1. Start the program.
2. Enter the number of locations.

3. For each location, enter:
    - Location name
    - Quantity received
    - Quantity distributed
    - Quantity damaged
    - Actual remaining stock
    - Minimum required stock
4. Calculate expected remaining stock:

Expected Stock = Received − Distributed − Damaged

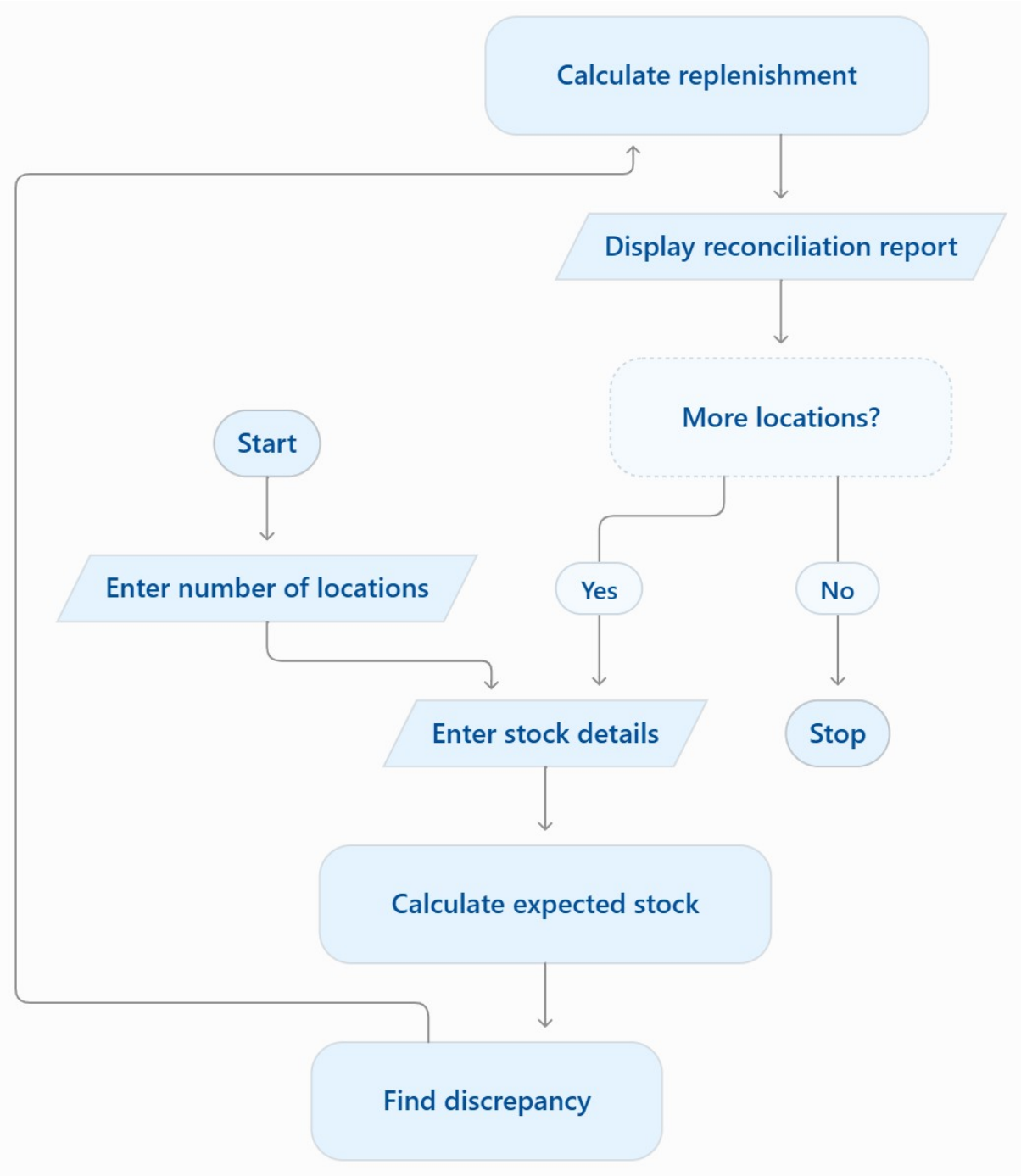
5. Compare expected stock with actual remaining stock.
6. Calculate discrepancy:

Discrepancy = Expected Stock − Actual Stock

7. Calculate replenishment required:

Replenishment = Minimum Stock − Actual Stock (if positive, otherwise 0).
8. Display the stock reconciliation report.
9. Stop.

**3. Flowchart**




**4.source code**

```
n = int(input("Enter number of locations: "))

for i in range(n):
    print("\nLocation", i + 1)
```

```
name = input("Enter location name: ")
received = int(input("Quantity received: "))
distributed = int(input("Quantity distributed: "))
damaged = int(input("Quantity damaged: "))
actual = int(input("Actual remaining stock: "))
minimum = int(input("Minimum required stock: "))

expected = received - distributed - damaged
discrepancy = expected - actual
replenish = max(0, minimum - actual)

print("\n--- Stock Report ---")
print("Location:", name)
print("Expected Stock:", expected)
print("Actual Stock:", actual)
print("Discrepancy:", discrepancy)

if discrepancy == 0:
    print("Status: Stock Matched")
elif discrepancy > 0:
    print("Status: Stock Shortage")
else:
    print("Status: Extra Stock")

print("Replenishment Required:", replenish)
```

**5.Sample Input**

```
Enter number of locations: 2

Location 1
Enter location name: Central Store
Quantity received: 1000
Quantity distributed: 600
Quantity damaged: 50
Actual remaining stock: 340
Minimum required stock: 400

Location 2
Enter location name: North Store
Quantity received: 800
Quantity distributed: 500
Quantity damaged: 20
Actual remaining stock: 280
Minimum required stock: 350
```

**6. Sample Output**

```
--- Stock Report ---
Location: Central Store
Expected Stock: 350
Actual Stock: 340
Discrepancy: 10
Status: Stock Shortage
Replenishment Required: 60

--- Stock Report ---
Location: North Store
Expected Stock: 280
Actual Stock: 280
Discrepancy: 0
Status: Stock Matched
Replenishment Required: 70
```

# Test Case 5

# Campus Event Financial Planning Using Python

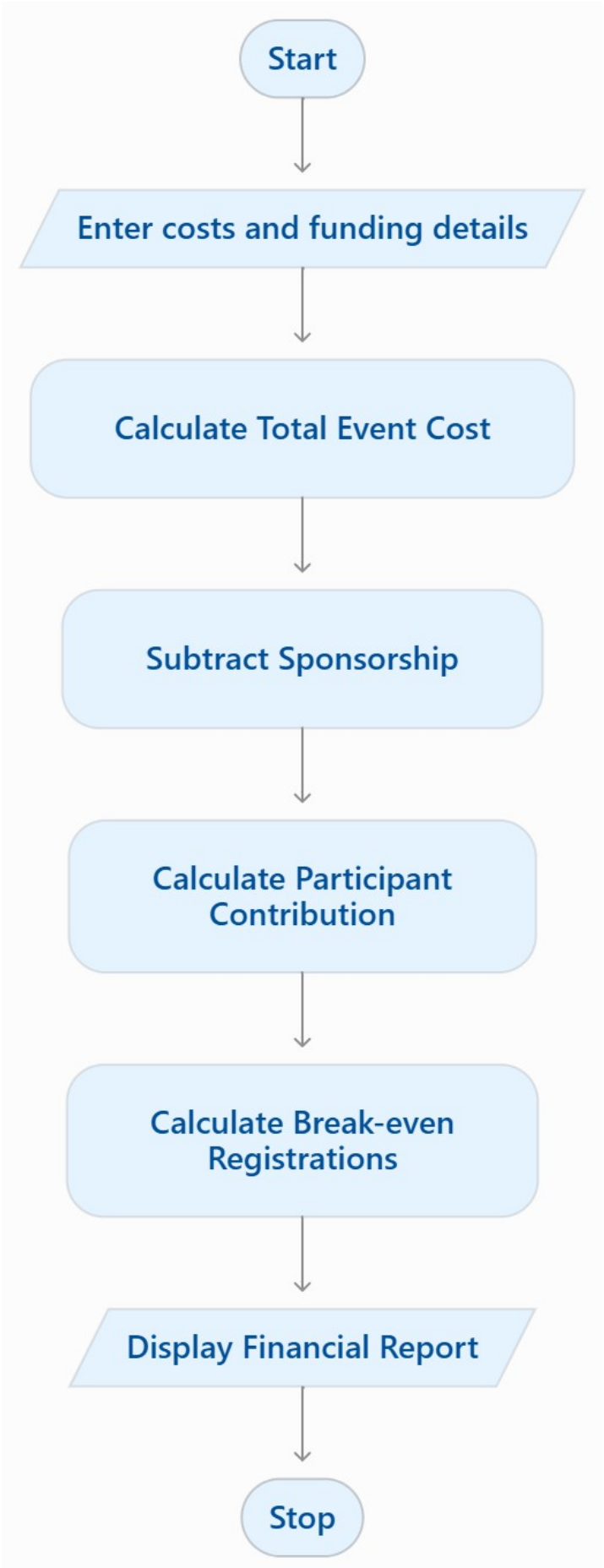
## 1. Problem Statement

To develop a Python program to calculate the total expenses of a campus event, including venue, equipment, catering, transport, and certificates. The program calculates the participant contribution, sponsorship shortfall, and minimum number of registrations required to break even.

## 2. Algorithm

1. Start.
2. Input venue, equipment, catering, transport, certificate costs, sponsorship amount, participant contribution, and expected participants.
3. Calculate the total event cost.
4. Calculate the amount remaining after sponsorship.
5. Calculate the required contribution per participant.
6. Calculate the minimum registrations needed to break even.
7. Display all costs and financial details.
8. Stop.

## 3. Flowchart


Start
Enter costs and funding details
Calculate Total Event Cost
Subtract Sponsorship
Calculate Participant Contribution
Calculate Break-even Registrations
Display Financial Report
Stop

**4.source code**

```
import math

try:
    venue = float(input("Enter venue cost: "))
    equipment = float(input("Enter equipment cost: "))
    catering = float(input("Enter catering cost per participant: "))
    transport = float(input("Enter transport cost: "))
    certificate = float(input("Enter certificate cost per participant: "))
    sponsorship = float(input("Enter sponsorship amount: "))
    contribution = float(input("Enter contribution per participant: "))
    participants = int(input("Enter expected participants: "))

    if min(venue, equipment, catering, transport,
           certificate, sponsorship, contribution) < 0:
        raise ValueError("Costs and funding cannot be negative")
    if participants <= 0 or contribution <= 0:
        raise ValueError("Participants and contribution must be positive")

    fixed_cost = venue + equipment + transport
    variable_cost = catering + certificate
    total_cost = fixed_cost + variable_cost * participants
    shortfall = max(0, total_cost - sponsorship)
    required_per_person = shortfall / participants

    if contribution > variable_cost:
        breakeven = max(0, math.ceil(
```

```
            (fixed_cost - sponsorship) /
            (contribution - variable_cost)
        ))
    else:
        breakeven = (
            0 if sponsorship >= fixed_cost else None
        )

    print("\n--- CAMPUS EVENT FINANCIAL REPORT ---")
    print("Total event cost: Rs.", round(total_cost, 2))
    print("Sponsorship: Rs.", round(sponsorship, 2))
    print("Sponsorship shortfall: Rs.", round(shortfall, 2))
    print("Required contribution per participant: Rs.",
          round(required_per_person, 2))
    print("Break-even registrations:",
          breakeven if breakeven is not None
          else "Not achievable at this contribution")

except ValueError as e:
    print("Invalid input:", e)
```

**5.sample input**

```
Enter venue cost: 10000
Enter equipment cost: 5000
Enter catering cost per participant: 200
Enter transport cost: 3000
Enter certificate cost per participant: 50
Enter sponsorship amount: 8000
Enter contribution per participant: 400
Enter expected participants: 100
```

**6.sample output**

```
--- CAMPUS EVENT FINANCIAL REPORT ---
Total event cost: Rs. 48000.0
Sponsorship: Rs. 8000.0
Sponsorship shortfall: Rs. 40000.0
Required contribution per participant: Rs. 400.0
Break-even registrations: 34
```

# Test Case 6

# Text-Based Invoice Data Extraction Using Python

**1. Problem Statement**

To develop a Python program that processes formatted invoice strings using basic string operations. The program extracts the product code, quantity, and amount from each invoice record and flags malformed records that do not follow the required format.

**2. Algorithm**

1.Start.

2.Input invoice records.

3.Split each record using commas.

4.Check whether the record contains exactly three fields.

5.Extract product code, quantity, and amount.

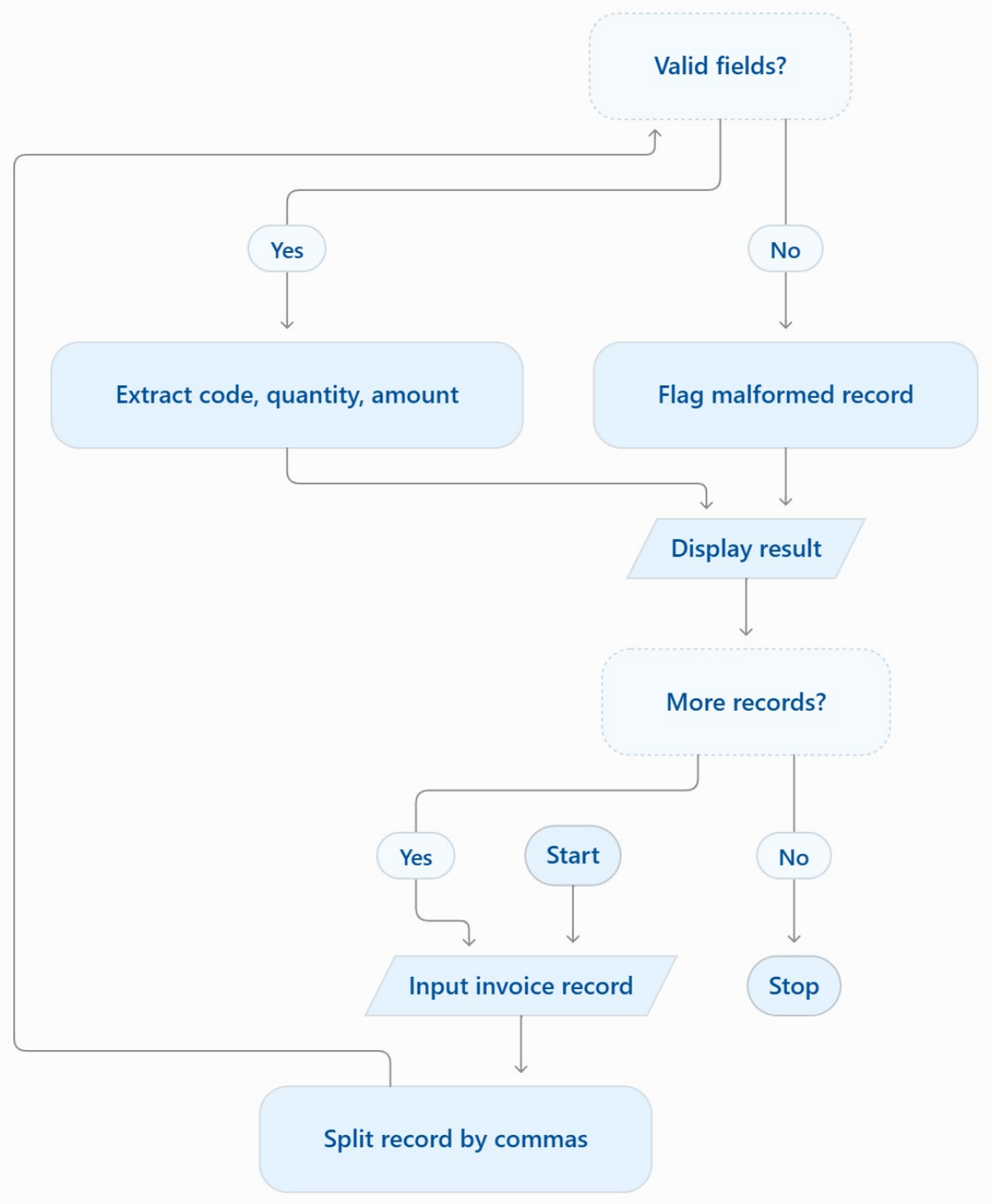
6.Validate the fields.

7.If valid, display the extracted data; otherwise, flag the record as malformed.

8.Repeat for all records.

9.Stop.

**3. Flowchart**




**4.source code**

```
records = [
    "P101,2,500.00",
    "P102,5,1250.50",
    "P103,abc,300",
```

```
    "P104,3",
    "P105,2,-100"
]

print("INVOICE DATA EXTRACTION")

for record in records:
    try:
        fields = record.split(",")

        if len(fields) != 3:
            raise ValueError

        code = fields[0].strip()
        quantity = int(fields[1].strip())
        amount = float(fields[2].strip())

        if not code or quantity <= 0 or amount < 0:
            raise ValueError

        print(f"Code: {code}, Quantity: {quantity}, "
              f"Amount: Rs. {amount:.2f}")

    except ValueError:
        print(f"Malformed record: {record}")
```

**5.sample input**

```
INVOICE DATA EXTRACTION
Code: P101, Quantity: 2, Amount: Rs. 500.00
Code: P102, Quantity: 5, Amount: Rs. 1250.50
Malformed record: P103,abc,300
Malformed record: P104,3
Malformed record: P105,2,-100
```

**6.Sample Output**

```
INVOICE DATA EXTRACTION
Code: P101, Quantity: 2, Amount: Rs. 500.00
Code: P102, Quantity: 5, Amount: Rs. 1250.50
Malformed record: P103,abc,300
Malformed record: P104,3
Malformed record: P105,2,-100
```

# Test Case 7

# Sports Tournament Ranking System Using Python

**1. Problem Statement**

To develop a Python program that stores sports tournament match results, calculates team points and tie-break values, and ranks teams according to a specified competition policy.

The program awards 3 points for a win, 1 point for a draw, and 0 points for a loss. Teams are ranked by total points first, then goal difference, and finally goals scored.

**2. Algorithm**

1.Start.

2.Input the number of teams and match results.

3.Initialize each team's points, goals scored, and goals conceded to zero.

4.Process each match and update team statistics.

5.Award 3 points for a win and 1 point to each team for a draw.

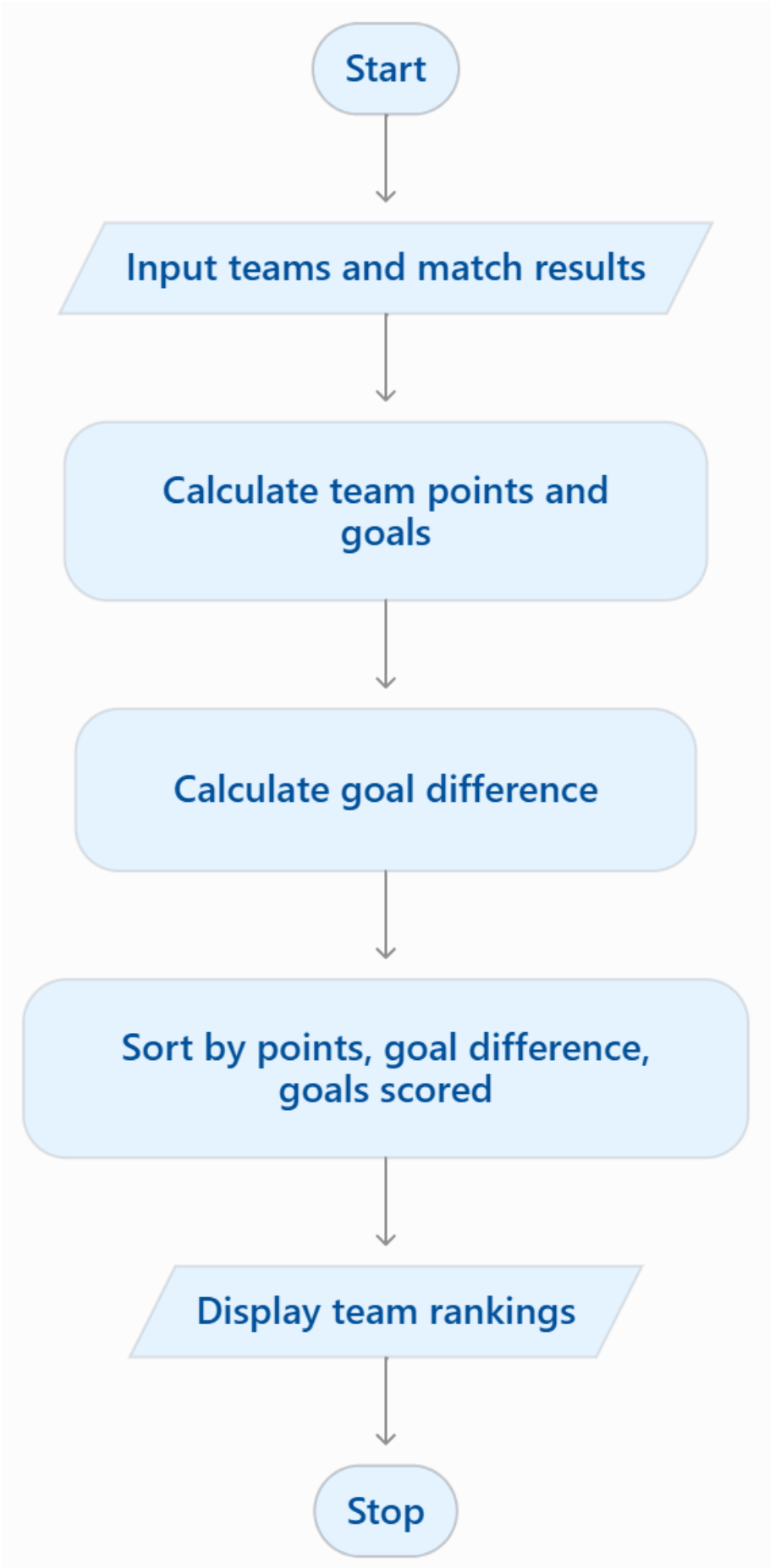
6.Calculate goal difference for each team.

7.Sort teams by points, goal difference, and goals scored in descending order.

8.Display the final tournament rankings.

9.Stop.

**3. Flowchart**


Start
Input teams and match results
Calculate team points and goals
Calculate goal difference
Sort by points, goal difference, goals scored
Display team rankings
Stop

**4.source code**

```
teams = {}

n = int(input("Enter number of teams: "))

for i in range(n):
    name = input("Enter team name: ")
    teams[name] = [0, 0, 0, 0]  # Points, GF, GA, GD

m = int(input("Enter number of matches: "))

for i in range(m):
    a, b = input("Enter teams and goals (A B GA GB): ").split()
    ga, gb = int(input("Enter goals for " + a + ": ")), int(input("Enter goals for " + b + ": "))

    teams[a][1] += ga
    teams[a][2] += gb
    teams[b][1] += gb
    teams[b][2] += ga

    if ga > gb:
        teams[a][0] += 3
    elif gb > ga:
        teams[b][0] += 3
    else:
        teams[a][0] += 1
        teams[b][0] += 1
```

```
for name in teams:
    teams[name][3] = teams[name][1] - teams[name][2]

ranking = sorted(teams.items(),
                 key=lambda x: (x[1][0], x[1][3], x[1][1]),
                 reverse=True)

print("\n--- TOURNAMENT RANKINGS ---")
print("Rank Team Points GF GA GD")

for i, (name, s) in enumerate(ranking, 1):
    print(i, name, s[0], s[1], s[2], s[3])
```

**5.Sample Input**

```
Enter number of teams: 3
Enter team name: A
Enter team name: B
Enter team name: C
Enter number of matches: 3
Enter teams and goals (A B GA GB): A B
Enter goals for A: 2
Enter goals for B: 1
Enter teams and goals (A B GA GB): B C
Enter goals for B: 1
Enter goals for C: 1
Enter teams and goals (A B GA GB): C A
Enter goals for C: 0
Enter goals for A: 2
```

**6.Sample Output**

```
--- TOURNAMENT RANKINGS ---
Rank Team Points GF GA GD
1 A 6 4 1 3
2 B 1 2 3 -1
3 C 1 1 3 -2
```

# Test Case 8

# Integrated Manufacturing Production Planner Using Python

**1. Problem Statement**

To develop a Python program that analyses available raw materials, production quantities, defect counts, and customer order demand. The program determines whether production is feasible, identifies material shortages, checks whether production can meet demand, and flags quality violations when defects exceed the permitted limit.

**2. Algorithm**

1. Start.
2. Input raw material available, material required per unit, production quantity, order demand, defect count, and maximum allowed defects.
3. Calculate the total material required.
4. Check whether available materials are sufficient.
5. Calculate good units by subtracting defective units from production quantity.
6. Compare good units with order demand.
7. Identify material shortages and quality violations.
8. Display the production feasibility report.
9. Stop.

**3. Flowchart**


Start
Input production data
Calculate material needs and good units
Materials sufficient?
No
Yes
Report material shortage
Check order demand and defects
Display production report
Stop

**4.source code**

```
try:
    available = int(input("Available raw material: "))
    required = int(input("Material required per unit: "))
    production = int(input("Planned production quantity: "))
    demand = int(input("Order demand: "))
    defects = int(input("Number of defective units: "))
    limit = float(input("Maximum defect rate (%): "))

    if min(available, required, production, demand, defects) < 0:
        raise ValueError("Values cannot be negative")
    if required == 0 or production == 0:
        raise ValueError("Material and production must be positive")
    if defects > production or not 0 <= limit <= 100:
        raise ValueError("Invalid defects or quality limit")

    material_needed = required * production
    shortage = max(0, material_needed - available)
    good_units = production - defects
    order_shortage = max(0, demand - good_units)
    defect_rate = defects / production * 100

    print("\n--- PRODUCTION REPORT ---")
    print("Material required:", material_needed)
    print("Material shortage:", shortage)
    print("Good units:", good_units)
    print("Order shortage:", order_shortage)
    print(f"Defect rate: {defect_rate:.2f}%")
```

```
    print("Production feasible:",
          "Yes" if shortage == 0 else "No")
    print("Quality status:",
          "Violation" if defect_rate > limit else "Acceptable")

except ValueError as e:
    print("Invalid input:", e)
```

**5.Sample Input**

```
Available raw material: 350
Material required per unit: 2
Planned production quantity: 200
Order demand: 195
Number of defective units: 8
Maximum defect rate (%): 3
```

**6.Sample Output**

```
--- PRODUCTION REPORT ---
Material required: 400
Material shortage: 50
Good units: 192
Order shortage: 3
Defect rate: 4.00%
Production feasible: No
Quality status: Violation
```

# Test Case 9

# Campus Festival Registration and Financial Audit Using Python

**1. Problem Statement**

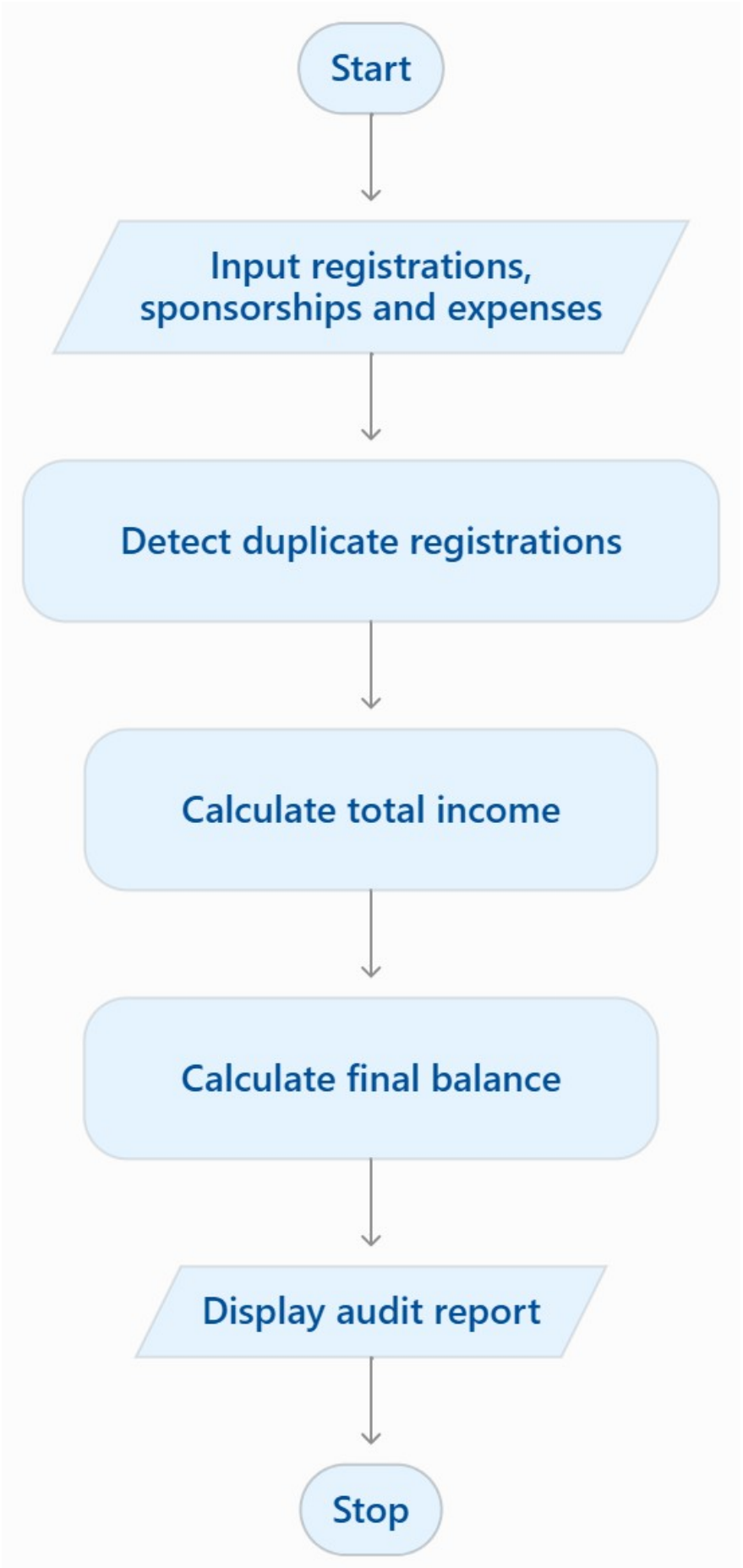
To develop a Python program that manages campus festival registrations and performs a financial audit. The program checks participant lists for duplicate

registrations, calculates total registration fees and sponsorship contributions, records festival expenses, and determines the final financial balance.

**2. Algorithm**

1. Start.
2. Input participant registration IDs and fees.
3. Identify and flag duplicate registration IDs.
4. Calculate total fees from valid registrations.
5. Input sponsorship contributions and festival expenses.
6. Calculate total income and total expenses.
7. Determine the final balance.
8. Display duplicate registrations and the financial audit report.
9. Stop.

**3. Flowchart**


Start
Input registrations, sponsorships and expenses
Detect duplicate registrations
Calculate total income
Calculate final balance
Display audit report
Stop

**4.source code**

```
participants = []
seen = set()
duplicates = []

n = int(input("Enter number of registrations: "))

for i in range(n):
    pid = input("Enter participant ID: ")
    fee = float(input("Enter registration fee: "))

    if pid in seen:
        duplicates.append(pid)
    else:
        seen.add(pid)
        participants.append(fee)

sponsorship = float(input("Enter total sponsorship: "))
expenses = float(input("Enter total expenses: "))

registration_total = sum(participants)
total_income = registration_total + sponsorship
balance = total_income - expenses

print("\n--- FESTIVAL FINANCIAL AUDIT ---")
print("Valid registrations:", len(participants))
print("Duplicate IDs:", duplicates)
print("Registration fees: Rs.", registration_total)
```

```
print("Sponsorship: Rs.", sponsorship)
print("Total income: Rs.", total_income)
print("Total expenses: Rs.", expenses)
print("Final balance: Rs.", balance)

if balance > 0:
    print("Status: Surplus")
elif balance < 0:
    print("Status: Deficit")
else:
    print("Status: Balanced")
```

**5.Sample Input**

```
Enter number of registrations: 4
Enter participant ID: P101
Enter registration fee: 500
Enter participant ID: P102
Enter registration fee: 500
Enter participant ID: P101
Enter registration fee: 500
Enter participant ID: P103
Enter registration fee: 500
Enter total sponsorship: 2000
Enter total expenses: 2500
```

**6. Sample Output**

```
--- FESTIVAL FINANCIAL AUDIT ---
Valid registrations: 3
Duplicate IDs: ['P101']
Registration fees: Rs. 1500.0
Sponsorship: Rs. 2000.0
Total income: Rs. 3500.0
Total expenses: Rs. 2500.0
Final balance: Rs. 1000.0
Status: Surplus
```

# Test Case 10

# University Scholarship Allocation System using Python

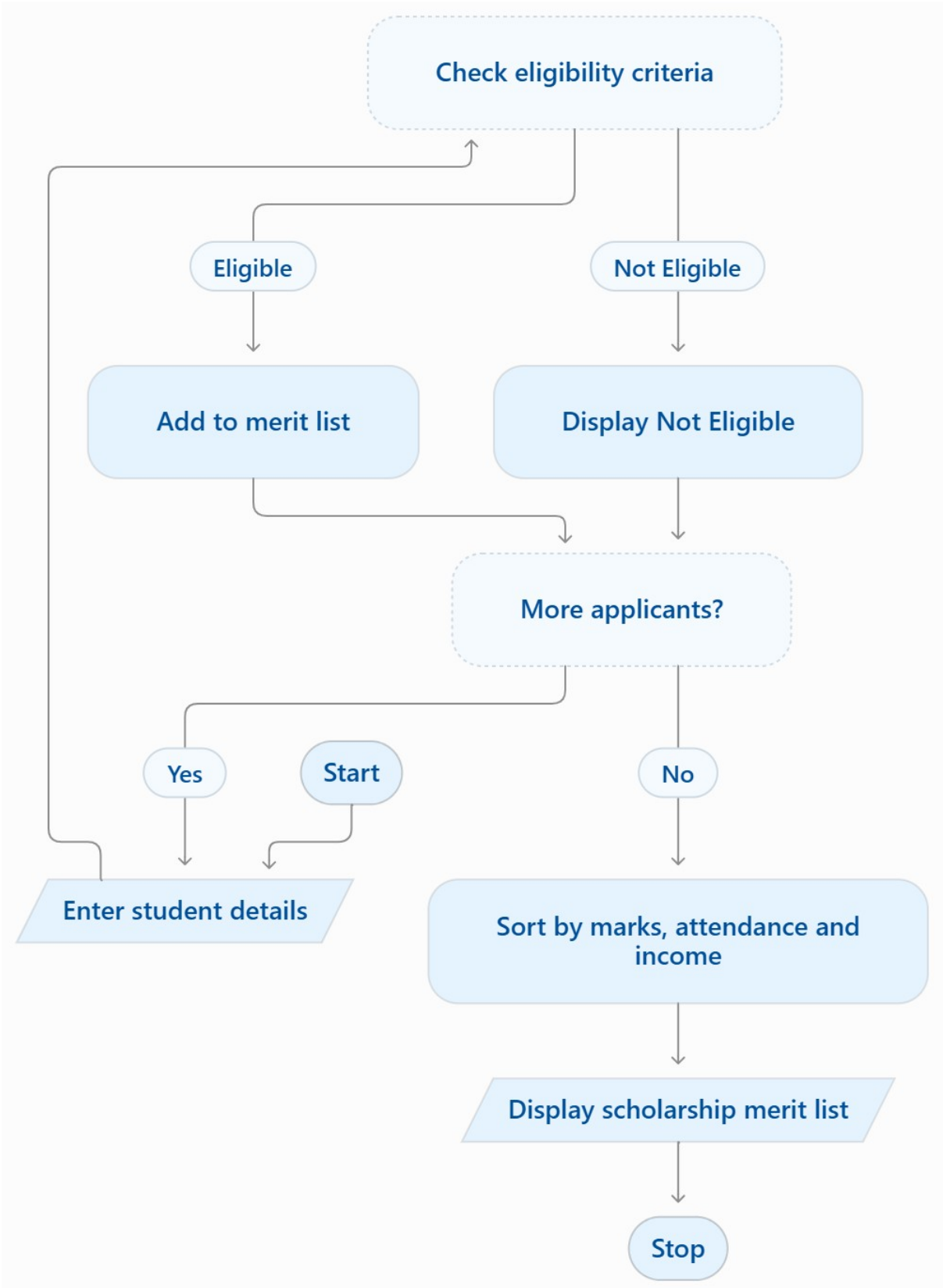
**1. Problem Statement**

The University Scholarship Allocation System evaluates students based on aggregate marks, subject-wise minimum marks, attendance percentage, and family annual income. It identifies eligible students and ranks them according to aggregate marks. If two students have equal marks, attendance is used as the first tie-breaker, followed by lower family income. The system displays the final ranked list of eligible students.

**2. Algorithm**

1. Start the program.
2. Read the number of applicants.
3. Enter each student's name, aggregate marks, subject marks, attendance percentage, and annual family income.
4. Check the eligibility criteria:
    - Aggregate marks must be at least 75%.
    - Every subject mark must be at least 60%.
    - Attendance must be at least 75%.
    - Annual family income must not exceed ₹2,50,000.
5. Add eligible students to the eligible list.
6. Sort eligible students by:
    - Higher aggregate marks first.
    - Higher attendance first if aggregate marks are equal.
    - Lower family income first if both are equal.
7. Display the ranked list of eligible students.
8. If no students qualify, display a suitable message.
9. Stop the program.

**3. Flowchart**




**4.source code**

students = []

```
n = int(input("Enter number of applicants: "))

for i in range(n):
    print("\nApplicant", i + 1)

    name = input("Enter student name: ")
    marks = float(input("Enter aggregate marks (%): "))
    subjects = list(map(float, input("Enter marks in 3 subjects: ").split()))
    attendance = float(input("Enter attendance (%): "))
    income = float(input("Enter annual family income: "))

    if (75 <= marks <= 100 and len(subjects) == 3
        and all(60 <= x <= 100 for x in subjects)
        and 75 <= attendance <= 100 and income >= 0
        and income <= 250000):

        students.append((name, marks, attendance, income))
        print("Eligible for scholarship")
    else:
        print("Not eligible for scholarship")

students.sort(key=lambda x: (-x[1], -x[2], x[3]))

print("\n--- SCHOLARSHIP MERIT LIST ---")

if students:
    for rank, s in enumerate(students, 1):
        print(f"{rank}. {s[0]} - Marks: {s[1]}%, "
```

```
            f"Attendance: {s[2]}%, Income: Rs.{s[3]}")
else:
    print("No eligible students found")
```

### 5.Sample Input

```
Applicant 1
Enter student name: Ravi
Enter aggregate marks (%): 85
Enter marks in 3 subjects: 80 90 75
Enter attendance (%): 90
Enter annual family income: 200000

Applicant 2
Enter student name: Sita
Enter aggregate marks (%): 85
Enter marks in 3 subjects: 75 80 70
Enter attendance (%): 95
Enter annual family income: 220000

Applicant 3
Enter student name: Aman
Enter aggregate marks (%): 70
Enter marks in 3 subjects: 65 70 75
Enter attendance (%): 80
Enter annual family income: 150000
```

### 6.Sample Output

```
Applicant 1
Eligible for scholarship

Applicant 2
Eligible for scholarship

Applicant 3
Not eligible for scholarship

--- SCHOLARSHIP MERIT LIST ---

1. Sita - Marks: 85.0%, Attendance: 95.0%, Income: Rs.220000.0

2. Ravi - Marks: 85.0%, Attendance: 90.0%, Income: Rs.200000.0
```